Abstraction & Polymorphism — Complete Deep Dive

---

1. ABSTRACTION — The Core Philosophy

What Does "Abstraction" Actually Mean?

Before writing a single line of code, you need to understand abstraction as a **life philosophy**, because that's literally what it is.

The definition is: **"The process of focusing on WHAT something does, while ignoring HOW it does that."**

Let's break this down with real life first.

The Real World Runs on Abstraction

Think about this chain:

You drive a car. You press the accelerator. The car speeds up. Do you know what happens inside? Fuel injectors spray fuel, pistons compress it, combustion happens, crankshaft rotates, power goes to wheels. You don't know any of this. You just pressed a pedal.

Now the car engineer who designed the engine — does he understand every physics principle from scratch? No. He uses known physics formulas and principles, abstracting away the deeper mathematics.

The physicist who derived those formulas — does he understand all of mathematics from first principles? No. He uses existing mathematical frameworks.

So the chain looks like:

```
You (driver)
   → abstracts away engine mechanics
Car Engineer
   → abstracts away physics derivations
Physicist
   → abstracts away pure mathematics
```

Mathematician
→ builds from axioms (the base)

If abstraction didn't exist, every person who wanted to drive a car would first need to become a mathematician → physicist → engineer → driver. That's impossible. Society would collapse.

The entire human civilization is built on layers of abstraction. Each layer hides the complexity of the layer below it and exposes only what's necessary.

Abstraction in OOP — The Two Statements

Abstraction in OOP says two things:

Statement 1: Represent only what is necessary.

When you model a real-world object in code, you don't represent EVERYTHING about it. You represent only what your application needs.

Example — modeling a Teacher:

- A teacher is a human being, made of cells, organs, bones, neurons
- But in your school management application, you don't care about cells
- You care about: `name`, `subject`, `salary` (data) and `teach()`, `takeAttendance()` (behavior)
- Everything else is ignored

This is abstraction — **represent whatever is necessary, ignore the rest.**

Statement 2: Even what you model, hide HOW it works.

Even for the things you DO model, you don't want everyone to understand the internal implementation. You just want them to be able to USE it.

Example — ATM Machine:

- You know it can `deposit()`, `withdraw()`, `checkBalance()`
- You do NOT know how money counting works internally, how the network call to your bank happens, how the security system works
- You just USE the interface provided

The ATM manufacturer gave you buttons and a screen (an interface). They hid everything else.

OOP Doesn't Simulate Reality — It Simulates Human PERCEPTION of Reality

This is a subtle but profound point from the video.

People say "OOP simulates the real world." That's a half-truth.

OOP simulates **what humans THINK about the real world** — not the exact reality.

When you think "car," you think:

- It drives
- It brakes
- It has a color

You don't think about 30,000 parts, metallurgical compositions, fluid dynamics. That's not how humans perceive a car.

So OOP models **human perception** — the abstracted view — not raw reality. This is why OOP is so natural and powerful. It matches how our minds actually work.

---

2. TWO TYPES OF ABSTRACTION IN JAVA

Java implements abstraction at two levels. This is where most people get confused. Let's be crystal clear.

TYPE 1: Low Level Abstraction

Definition: Hiding implementation details. The user of your code doesn't need to know HOW a method works, just WHAT it does.

This is implemented through **regular classes**.

```
class Car {
    void start() {
        // Complex internal logic:
        // - Check fuel level
        // - Engage starter motor
        // - Fire ignition system
        // - Monitor RPM
        System.out.println("Car started");
```

```
    }

    void accelerate() {
        // Complex internal logic
        System.out.println("Car accelerating");
    }

    void brake() {
        // Complex internal logic
        System.out.println("Car braking");
    }
}
```

When someone uses this class:

```
Car c = new Car();
c.start();      // They have NO IDEA what happens inside
c.accelerate(); // They just call it and it works
c.brake();      // Implementation is hidden
```

The caller doesn't open the `Car.java` file. They just use the object. The implementation is hidden. That's low level abstraction.

This is already happening when you write any class. You've been doing low level abstraction from day one without realizing it.

The problem with low level abstraction is that **WHAT and HOW are tightly coupled.**

When you write `c.start()`, it ALWAYS calls THIS specific `start()` method. The what (starting the car) and the how (this specific start implementation) are locked together.

c.start() → always calls → this exact start() code
c.brake() → always calls → this exact brake() code

What if tomorrow you need an ElectricCar with completely different internal workings? The what is the same (start a car) but the how is completely different (no combustion engine).

That's where High Level Abstraction comes in.

TYPE 2: High Level Abstraction

Definition: Separating WHAT something does FROM HOW it does it.

The WHAT and HOW become decoupled. The same "what" can have multiple different "hows."

The Problem it Solves:

```
class FuelCar {
    void drive() {
        // Combustion engine logic
        System.out.println("Fuel car driving");
    }
}

class ElectricCar {
    void drive() {
        // Electric motor logic
        System.out.println("Electric car driving");
    }
}

// In your main code:
FuelCar fc = new FuelCar();
ElectricCar ec = new ElectricCar();

fc.drive(); // Always fuel car's drive
ec.drive(); // Always electric car's drive
```

Here, `fc` is tightly coupled to `FuelCar`. `ec` is tightly coupled to `ElectricCar`. You need separate variables, separate types, separate everything.

Tomorrow you add DieselCar, WaterCar, HydrogenCar — you keep multiplying variables and types.

The Solution:

What if one reference variable `c` of type `Car` could point to ANY type of car? The WHAT (drive a car) stays the same. The HOW (combustion vs electric) is decided at runtime.

```
Car c = new ElectricCar(); // c points to electric car
c.drive();                 // calls ElectricCar's drive

c = new FuelCar();         // c now points to fuel car
c.drive();                 // calls FuelCar's drive
```

Same variable `c`. Same method call `c.drive()`. Different actual behavior depending on what object `c` points to.

This is high level abstraction — separating WHAT from HOW.

Java implements this through **two mechanisms:**

1. Abstract Classes
2. Interfaces

---

3. ABSTRACT CLASSES — Complete Deep Dive

Why Abstract Classes Exist

Let's trace the thought process that leads to needing abstract classes.

You have `FuelCar` and `ElectricCar`. Both are types of cars. Using inheritance, you create a parent class `Car`:

```
class Car {
    void start() {
        System.out.println("Generic car starting"); // What do you write here?
    }

    void accelerate() {
        System.out.println("Car accelerating at 10 km/h"); // Meaningless!
    }

    void brake() {
        System.out.println("Car stopping"); // Too generic
    }
}

class FuelCar extends Car {
    // Inherits start, accelerate, brake
    // But its accelerate is completely different!
}

class ElectricCar extends Car {
    // Also completely different acceleration!
}
```

The problem: What do you write inside `accelerate()` in `Car`? A car is just an idea, a concept. "Car" doesn't have a specific acceleration mechanism — FuelCar does, ElectricCar does, but "Car" in general? You'd just write some meaningless generic code.

The solution: Don't write anything in `accelerate()` for `Car`. Just DECLARE it — say "every car CAN accelerate, but I'm not going to define HOW here. Each specific car type will define HOW."

This is exactly what an abstract method is:

```
abstract class Car {
    // Fully implemented - every car starts the same way (hypothetically)
    void start() {
        System.out.println("Car started");
    }

    // Declared but NOT implemented - HOW is left to subclasses
    abstract void accelerate();

    // Also declared but NOT implemented
    abstract void brake();
}
```

The `abstract` keyword on a method means: **"I'm declaring this method exists, but I'm NOT providing the implementation. Whoever extends this class MUST provide the implementation."**

The Abstract Class Rule: If any method is abstract, the class MUST be abstract

Why? Simple logic:

If `accelerate()` has no implementation, and you create a `Car` object:

```
Car c = new Car(); // If this were allowed...
c.accelerate();    // What would happen? There's nothing to call!
```

Java prevents this by saying: **if a class has even ONE abstract method, the class itself must be declared abstract, AND you cannot create objects of that class.**

```
abstract class Car {    // MUST be abstract because it has abstract methods
```

```
    abstract void accelerate();
    abstract void brake();
    void start() { System.out.println("Started"); }
}

Car c = new Car(); // COMPILE ERROR: Cannot instantiate abstract class
```

You can only create objects of CONCRETE classes (classes that implement all abstract methods).

The Contract: Child Classes MUST Implement All Abstract Methods

When a class extends an abstract class, it has a contract: **implement ALL abstract methods, or be abstract yourself.**

```
abstract class Car {
    abstract void accelerate();
    abstract void brake();
    void start() { System.out.println("Car started"); }
}

class FuelCar extends Car {
    @Override
    void accelerate() {
        // Combustion engine logic
        System.out.println("Fuel car accelerating - combustion engine engaged");
    }

    @Override
    void brake() {
        // Hydraulic brake logic
        System.out.println("Fuel car braking - hydraulic brakes applied");
    }
    // Inherits start() from Car
}

class ElectricCar extends Car {
    @Override
    void accelerate() {
        // Electric motor logic
        System.out.println("Electric car accelerating - electric motor spinning");
    }
```

```
    @Override
    void brake() {
        // Regenerative braking - actually charges the battery!
        System.out.println("Electric car braking - regenerative braking, charging battery");
    }
    // Also inherits start() from Car
}
```

If `FuelCar` doesn't implement `accelerate()`, Java gives a compile error: **"FuelCar must implement the inherited abstract method Car.accelerate()"**

Unless `FuelCar` itself is abstract — then its subclass must implement it.

Using Abstract Classes — The Power of High Level Abstraction

```
// Now in main code:
Car c = new ElectricCar(); // Car reference, ElectricCar object

c.start();      // Calls Car's start() - "Car started"
c.accelerate(); // Calls ElectricCar's accelerate()
c.brake();      // Calls ElectricCar's brake()

// Change just ONE LINE:
c = new FuelCar(); // Now points to FuelCar

c.start();      // Still calls Car's start() - same output
c.accelerate(); // Now calls FuelCar's accelerate() - DIFFERENT behavior
c.brake();      // Now calls FuelCar's brake() - DIFFERENT behavior
```

The WHAT (`c.accelerate()`) is separated from the HOW (which specific accelerate runs). The HOW is determined at runtime based on what object `c` points to.

You've successfully separated WHAT from HOW. This is high level abstraction.

Complete Code Example

```
abstract class Car {
    private String brand;
    private int year;
```

```
    public Car(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    // Concrete method - same for all cars
    public void start() {
        System.out.println(brand + " (" + year + ") is starting...");
    }

    // Abstract methods - each car type implements differently
    public abstract void accelerate();
    public abstract void brake();

    // Getter
    public String getBrand() { return brand; }
}

class FuelCar extends Car {
    private int fuelLevel;

    public FuelCar(String brand, int year, int fuelLevel) {
        super(brand, year);
        this.fuelLevel = fuelLevel;
    }

    @Override
    public void accelerate() {
        if (fuelLevel <= 0) {
            System.out.println("Cannot accelerate - no fuel!");
            return;
        }
        fuelLevel--;
        System.out.println(getBrand() + " accelerating - combustion engine roaring!");
    }

    @Override
    public void brake() {
        System.out.println(getBrand() + " braking - hydraulic disc brakes engaged");
    }
}

class ElectricCar extends Car {
```

```
    private int batteryLevel;

    public ElectricCar(String brand, int year, int batteryLevel) {
        super(brand, year);
        this.batteryLevel = batteryLevel;
    }

    @Override
    public void accelerate() {
        if (batteryLevel <= 0) {
            System.out.println("Cannot accelerate - battery dead!");
            return;
        }
        batteryLevel--;
        System.out.println(getBrand() + " accelerating - electric motor humming silently");
    }

    @Override
    public void brake() {
        batteryLevel++; // Regenerative braking charges battery!
        System.out.println(getBrand() + " braking - regenerative braking, battery charging!");
    }
}

// Main
public class Main {
    public static void main(String[] args) {
        // High level abstraction in action:
        Car c = new ElectricCar("Tesla", 2024, 80);

        c.start();      // "Tesla (2024) is starting..."
        c.accelerate(); // "Tesla accelerating - electric motor humming silently"
        c.brake();      // "Tesla braking - regenerative braking, battery charging!"

        System.out.println("---");

        c = new FuelCar("Toyota", 2023, 50); // Just change this ONE LINE

        c.start();      // "Toyota (2023) is starting..."
        c.accelerate(); // "Toyota accelerating - combustion engine roaring!"
        c.brake();      // "Toyota braking - hydraulic disc brakes engaged"

        // c = new Car("Generic", 2024); // COMPILE ERROR!
    }
```

```
}
```

---

4. INTERFACES — Complete Deep Dive

What is an Interface?

An interface is often confused with abstract classes because the syntax looks similar. But their PURPOSE is fundamentally different.

Critical Rule to memorize: An interface has **NOTHING to do with objects.** A class is a blueprint for an object. An interface is NOT a blueprint for an object.

An interface is a **CONTRACT.**

Let me explain contract through real life first.

When you take a bank loan, you sign a contract. The contract says:

- You MUST repay ₹X every month
- You MUST pay interest at Y%
- If you default, Z will happen

The bank doesn't care HOW you earn the money. Work a job, start a business, sell assets — they don't care. You just MUST fulfill the contract terms.

An interface works exactly like this. It says: **"If you want to be considered a [thing], you MUST implement these methods. HOW you implement them is entirely your choice."**

Interface Syntax

```
interface Car {
   void start();      // Declared only, NOT implemented
   void accelerate(); // Declared only, NOT implemented
   void brake();      // Declared only, NOT implemented
}
```

Key differences from abstract class:

- Keyword is `interface`, not `class`
- NO `abstract` keyword needed on methods — **all methods are abstract by default**

- Class uses `implements`, not `extends`, to use an interface

```
class FuelCar implements Car {
    @Override
    public void start() {
        System.out.println("Fuel car starting");
    }

    @Override
    public void accelerate() {
        System.out.println("Fuel car accelerating");
    }

    @Override
    public void brake() {
        System.out.println("Fuel car braking");
    }
}

class ElectricCar implements Car {
    @Override
    public void start() { ... }

    @Override
    public void accelerate() { ... }

    @Override
    public void brake() { ... }
}
```

Notice: `extends` for classes, `implements` for interfaces. This is intentional — you EXTEND a class (build upon it), you IMPLEMENT an interface (fulfill a contract).

The `public` Rule in Interfaces

All methods in an interface are **public by default.** When you implement them in a class, you CANNOT reduce their visibility. You can only keep them public or make them more visible (which doesn't apply here since public is already maximum).

```
interface Car {
    void start(); // Implicitly: public abstract void start()
```

```
}

class FuelCar implements Car {
    public void start() { ... }  // CORRECT - public matches
    void start() { ... }         // ERROR - reduces visibility to default
    private void start() { ... } // ERROR - reduces to private
}
```

Error message: "Cannot reduce the visibility of the inherited method from Car."

Why Interfaces? The Real Purpose — Roles and Responsibilities

Here's the deep insight: **Interfaces define ROLES, not families.**

Abstract classes define families of similar objects (all cars belong to the same family). Interfaces define what roles/capabilities something has.

The naming convention tells you everything. Look at Java's own interfaces:

- `Comparable` — can this object be compared?
- `Serializable` — can this object be serialized?
- `Runnable` — can this object run in a thread?
- `Iterable` — can this object be iterated?

The `-able` suffix means "capable of." An interface says **"if you want to be CAPABLE of this, implement these methods."**

The Flyable Example:

```
interface Flyable {
    void fly();
    void land();
}

class Bird implements Flyable {
    @Override
    public void fly() {
        System.out.println("Bird flapping wings to fly");
    }

    @Override
    public void land() {
```

```
        System.out.println("Bird gliding down to perch");
    }
}

class Airplane implements Flyable {
    @Override
    public void fly() {
        System.out.println("Airplane using aerodynamics and jet engines");
    }

    @Override
    public void land() {
        System.out.println("Airplane extending landing gear, touching down");
    }
}

class Superman implements Flyable {
    @Override
    public void fly() {
        System.out.println("Superman using superpowers to fly");
    }

    @Override
    public void land() {
        System.out.println("Superman landing with a boom");
    }
}
```

`Bird`, `Airplane`, `Superman` — three completely unrelated things. They don't belong to the same family. But they all share the ROLE of being flyable.

An abstract class couldn't represent this because you can't make `Bird`, `Airplane`, and `Superman` all extend the same parent class — they're too different.

But an interface perfectly captures the shared capability: "if you can fly, implement `Flyable`."

Multiple Interface Implementation — The Killer Feature

Remember from the inheritance chapter — Java doesn't support multiple inheritance with classes because of the Diamond Problem. But **Java supports multiple interface implementation.**

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

interface Runnable {
    void run();
}

// A duck can fly, swim, AND run!
class Duck implements Flyable, Swimmable, Runnable {
    @Override
    public void fly() {
        System.out.println("Duck flying awkwardly");
    }

    @Override
    public void swim() {
        System.out.println("Duck paddling through water");
    }

    @Override
    public void run() {
        System.out.println("Duck waddling on land");
    }
}
```

A `Duck` has THREE roles: it can fly, swim, and run. You can't express this with abstract classes. You can't extend three parent classes. But you can implement three interfaces.

Why doesn't the Diamond Problem occur with interfaces?

With abstract classes, the problem was: two parent classes both have an implementation of `show()`, and the child doesn't know which to call.

With interfaces, there IS no implementation (in traditional Java). Every interface just declares `fly()` without implementing it. The child class provides the implementation. There's only ONE implementation of `fly()` — the one in `Duck`. No ambiguity.

```
interface A { void show(); }  // No implementation
```

```
interface B { void show(); }  // No implementation

class C implements A, B {
    public void show() {  // ONE implementation, no conflict
        System.out.println("C's show");
    }
}
```

No diamond problem because interfaces don't carry implementations to conflict with each other.

Interfaces Cannot Be Instantiated Either

```
Car c = new Car(); // ERROR - interface cannot be instantiated
```

But you can use interface as a reference type:

```
Car c = new FuelCar();     // Valid - FuelCar implements Car
Car c2 = new ElectricCar(); // Valid - ElectricCar implements Car
```

The same high-level abstraction works:

```
Car c = new ElectricCar();
c.start();      // ElectricCar's start
c.accelerate(); // ElectricCar's accelerate

c = new FuelCar(); // Now points to FuelCar
c.start();      // FuelCar's start
c.accelerate(); // FuelCar's accelerate
```

Default Methods in Interfaces (Modern Java)

In older Java, interfaces could ONLY have abstract methods. In Java 8+, interfaces can have `default` methods — methods with implementation.

```
interface Car {
    void accelerate(); // Still abstract
    void brake();      // Still abstract

    // Default method - has implementation
```

```
    default void start() {
        System.out.println("Car starting via default method");
    }
}
```

This was added to allow adding new methods to existing interfaces without breaking all existing implementations. If you add a new abstract method to an interface, every class that implements it would break (compile error). With `default`, existing classes aren't affected.

---

5. ABSTRACT CLASS vs INTERFACE — When to Use Which

This is the question everyone struggles with. Here's the definitive answer.

Use Abstract Class when:

You're modeling a FAMILY of related objects that share some common implementation.

- All cars share the same `start()` implementation → put it in abstract `Car` class
- All bank accounts share the same interest calculation → put it in abstract `BankAccount` class
- All animals share the same `breathe()` implementation → put it in abstract `Animal` class

The key test: **Is there shared CODE (implementation) among the subclasses?** If yes, abstract class.

```
abstract class Shape {
    String color;

    // Shared implementation - all shapes have color
    void setColor(String color) {
        this.color = color;
    }

    // Abstract - each shape calculates area differently
    abstract double getArea();
    abstract double getPerimeter();
}
```

Use Interface when:

You're defining a ROLE or CAPABILITY that unrelated classes might share.

- Things that can fly (Bird, Airplane, Superman — unrelated)
- Things that can be compared (String, Integer, Date — unrelated)
- Things that can be serialized (File, Object — different families)

The key test: **Are the implementing classes from completely different families?** If yes, interface.

```
interface Printable {
    void print();
}

class Document implements Printable {
    public void print() { System.out.println("Printing document"); }
}

class Photo implements Printable {
    public void print() { System.out.println("Printing photo"); }
}

class Invoice implements Printable {
    public void print() { System.out.println("Printing invoice"); }
}
```

The Summary Table:

Feature	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Inheritance keyword	`extends`	`implements`
Can have implemented methods	Yes	Only `default` methods (Java 8+)
Can have fields/variables	Yes	Only `public static final` constants
Multiple inheritance	No (only one extends)	Yes (multiple implements)
Constructor	Yes	No

Represents	Family of objects	Roles/Capabilities/Contract
Access modifiers on methods	Any	Only `public`
When to use	Shared code + is-a relationship	Unrelated classes sharing behavior

---

6. ABSTRACTION vs ENCAPSULATION — The Critical Difference

This is the interview question that trips everyone up. They look similar but are fundamentally different.

Encapsulation is about DATA SECURITY

Encapsulation says:

1. Bundle data and behavior together
2. **Restrict access to sensitive data** using access modifiers

The goal is **protection**. You make balance `private` because you don't want anyone directly modifying it. The data is protected. Without a setter or controlled method, no one can change it.

```
class BankAccount {
   private double balance; // PROTECTED - no unauthorized access

   public void deposit(double amount) {
      if (amount > 0) balance += amount; // Controlled access
   }
}
```

You CAN'T access `balance` even if you want to. It's locked.

Abstraction is about DATA/IMPLEMENTATION HIDING

Abstraction says: **hide complexity so users don't need to understand it.**

The goal is **simplicity**. You hide the internal implementation of `accelerate()` not for security, but because the user doesn't NEED to know it. They just need to know WHAT it does.

```
class Car {
    void accelerate() {
        // Complex combustion logic
        // User doesn't know or care about this
        // It's hidden for simplicity, not security
    }
}
```

You CAN look at the implementation if you want to (open the source file). It's not locked. It's just hidden to reduce complexity.

The Key Distinction:

Encapsulation: "You CANNOT access this data even if you try. It's locked for security."

Abstraction: "You don't NEED to know how this works. It's hidden for simplicity. You can find out if you really want to, but why would you?"

	Encapsulation	Abstraction
Goal	Data security	Simplicity / Hide complexity
Mechanism	Access modifiers (private, public)	Abstract classes, Interfaces, Regular classes
Nature of hiding	MANDATORY - you literally cannot access	OPTIONAL - you could look but don't need to
What's hidden	Data (fields/variables)	Implementation details (how methods work)
Real world analogy	Bank vault — you physically cannot access without permission	Car engine — you CAN open the hood, you just don't need to

---

7. POLYMORPHISM — Complete Deep Dive

What Does "Polymorphism" Mean?

Breaking it down:

- **Poly** = Many
- **Morphism** = Forms

Polymorphism = **Many Forms**

In real life: the same stimulus applied to the same kind of thing produces different responses based on context.

Real life example 1 — Same object, same command, different context:

A human being. You tell them: "RUN."

- Normal situation: they run at 2 km/h
- A dog is chasing them: they run at 5 km/h

Same object (human), same command (run), different parameter (dog present/not present) → different response.

Real life example 2 — Different objects, same command:

Three animals: Dog, Duck, Human. You tell all three: "RUN."

- Dog runs on four legs
- Duck waddles awkwardly
- Human runs on two legs

Different objects, same command, same situation → each responds differently based on their nature.

These two scenarios map to the **two types of polymorphism** in Java.

---

8. COMPILE TIME POLYMORPHISM (Static Polymorphism)

Maps to real life example 1: Same object, same command, different parameter → different response.

Implemented through: Method Overloading

What is Method Overloading?

Same method name, different parameters (different number, different types, or different order). The compiler decides at COMPILE TIME which version to call based on the arguments you pass.

```
class Human {
```

```
    // Version 1: Normal situation
    void run() {
        System.out.println("Running at 2 km/h - normal pace");
    }

    // Version 2: Dog is chasing
    void run(boolean isDogBehind) {
        if (isDogBehind) {
            System.out.println("Running at 5 km/h - PANIC MODE");
        }
    }

    // Version 3: Running with specific speed target
    void run(int targetSpeed) {
        System.out.println("Running at " + targetSpeed + " km/h");
    }

    // Version 4: Different parameter types
    void run(String destination) {
        System.out.println("Running towards " + destination);
    }
}

Human h = new Human();

h.run();            // Calls version 1 - no parameters
h.run(true);        // Calls version 2 - boolean parameter
h.run(8);           // Calls version 3 - int parameter
h.run("home");      // Calls version 4 - String parameter
```

Why is it COMPILE TIME?

The compiler can determine WHICH version of `run()` to call just by looking at your code. It doesn't need to run the program.

```
h.run();     // No args → definitely version 1 → decided at compile time
h.run(true); // Boolean arg → definitely version 2 → decided at compile time
```

The decision is made before the program ever runs. That's why it's called **compile time polymorphism.**

It's also called **Static Polymorphism** because the binding (which method to call) is static — fixed at compile time.

Rules for Method Overloading:

```
class Calculator {
    // Valid overloads - different parameter TYPES
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }

    // Valid overload - different NUMBER of parameters
    int add(int a, int b, int c) { return a + b + c; }

    // Valid overload - different ORDER of parameter types
    void print(String s, int n) { System.out.println(s + n); }
    void print(int n, String s) { System.out.println(n + s); }

    // INVALID - return type alone is NOT enough to distinguish
    // int add(int a, int b) { return a + b; }      // ERROR - same signature
    // double add(int a, int b) { return a + b; }   // ERROR - same signature
}
```

Return type alone is NOT enough to create an overload. The parameter list must differ.

---

9. RUNTIME POLYMORPHISM (Dynamic Polymorphism)

Maps to real life example 2: Different objects, same command, each responds differently.

Implemented through: Method Overriding

What is Method Overriding?

A child class provides its own implementation of a method that already exists in the parent class. The method name, return type, and parameters are exactly the same — only the body (implementation) differs.

```
abstract class Animal {
    abstract void run(); // What - declared but not implemented
}

class Dog extends Animal {
    @Override
    public void run() {
        System.out.println("Dog running on four legs at high speed");
    }
```

```
}

class Duck extends Animal {
    @Override
    public void run() {
        System.out.println("Duck waddling awkwardly");
    }
}

class Human extends Animal {
    @Override
    public void run() {
        System.out.println("Human running on two legs");
    }
}
```

Why is it RUNTIME?

Because WHICH `run()` gets called is determined at runtime — when the program is actually executing.

```
Animal a; // Just a reference, no object yet

// Suppose user types "dog" as input
String userInput = scanner.nextLine();

if (userInput.equals("dog")) {
    a = new Dog();    // Object created at runtime
} else if (userInput.equals("duck")) {
    a = new Duck();   // Object created at runtime
} else {
    a = new Human();  // Object created at runtime
}

a.run(); // Which run()? DECIDED AT RUNTIME based on what was input
```

The compiler can't know at compile time which `run()` will be called because it doesn't know what the user will type. The decision happens at runtime. That's why it's **runtime polymorphism.**

It's also called **Dynamic Polymorphism** because the binding is dynamic — decided while the program runs.

How Runtime Polymorphism Works Internally:

```
Animal a = new Dog(); // a is Animal reference, Dog object
a.run(); // Which run()?
```

Java looks at the **actual object** in memory (Dog), not the **reference type** (Animal), to decide which method to call. This is called **dynamic dispatch.**

```
Reference type: Animal
Actual object:  Dog
Method call:    a.run()
Java says:      "What is the actual object? Dog. Does Dog have run()? Yes. Call Dog's run()."
```

Full Runtime Polymorphism Example:

```
abstract class Shape {
    abstract double area();

    void printArea() {
        System.out.println("Area = " + area()); // Calls the appropriate area()
    }
}

class Circle extends Shape {
    double radius;

    Circle(double radius) { this.radius = radius; }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double width, height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
```

```
    double area() {
        return width * height;
    }
}

class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    double area() {
        return 0.5 * base * height;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(3, 8)
        };

        for (Shape s : shapes) {
            s.printArea(); // Each calls its own area() - runtime polymorphism!
        }
        // Output:
        // Area = 78.53...
        // Area = 24.0
        // Area = 12.0
    }
}
```

The `for` loop calls `printArea()` on each shape. `printArea()` calls `area()`. Which `area()` runs? Depends on the actual object type at runtime. That's dynamic polymorphism in action.

---

10. STATIC METHODS AND POLYMORPHISM

Static Methods Are NOT Overridden — They Are Hidden

```
class A {
   static void fun() {
      System.out.println("Hello from A");
   }
}

class B extends A {
   static void fun() {
      System.out.println("Bye from B");
   }
}

public class Main {
   public static void main(String[] args) {
      A a = new B(); // A reference, B object
      a.fun();       // Prints "Hello from A" ← NOT B's fun!
   }
}
```

This surprises people. The object is of type B. Shouldn't B's `fun()` run? With non-static methods, yes. But static methods are different.

Why?

Static methods belong to the CLASS, not to objects. When Java sees `a.fun()`, it looks at the TYPE of the reference (`A`), not the actual object. Since `fun()` is static, it's bound to class `A`. The object type doesn't matter.

This is called **method hiding** (not overriding) for static methods.

Non-static: Java looks at ACTUAL OBJECT type → dynamic dispatch → runtime decision
Static: Java looks at REFERENCE type → static binding → compile time decision

Practical Implication:

```
A a = new B();
a.fun();        // A's fun() - because reference is type A

B b = new B();
b.fun();        // B's fun() - because reference is type B

// Even though both a and b point to B objects, different fun() is called
```

// because reference types differ

With static methods, the reference type controls which method runs, not the object type.

---

11. PRIVATE METHODS AND POLYMORPHISM

Private Methods Cannot Be Overridden

```
class A {
    private void secret() {
        System.out.println("A's secret");
    }

    void reveal() {
        secret(); // Calls A's secret()
    }
}

class B extends A {
    private void secret() { // This is NOT overriding A's secret
        System.out.println("B's secret");
    }

    // B has its OWN secret() that has nothing to do with A's secret()
}

A a = new B();
a.reveal(); // Prints "A's secret" - NOT B's secret
```

Why? Because `A`'s `secret()` is private. B can't even SEE it, let alone override it. B's `secret()` is a completely separate, independent method that happens to have the same name.

When `reveal()` calls `secret()`, it calls `A`'s `secret()` because that's the only `secret()` A knows about.

The rule: You can only override methods you can ACCESS. Private methods are invisible to child classes, so they cannot be overridden.

---

12. FINAL METHODS, FINAL CLASSES AND POLYMORPHISM

Final Methods Cannot Be Overridden

```
class A {
    final void cantTouchThis() {
        System.out.println("A's method - FINAL, cannot override");
    }

    void canOverride() {
        System.out.println("A's method - can be overridden");
    }
}

class B extends A {
    // void cantTouchThis() { } // COMPILE ERROR!
    // Error: "Cannot override the final method from A"

    @Override
    void canOverride() { // This is fine
        System.out.println("B overrides this");
    }
}
```

When do you use `final` on a method? When you have a method that implements critical business logic and you want to guarantee that no subclass can change its behavior.

Example:

```
class PaymentProcessor {
    // This security check must never be bypassed or overridden
    final boolean validatePayment(double amount) {
        // Critical security logic
        // No subclass should be able to override this
        return amount > 0 && amount < 1000000;
    }
}
```

Final Classes Cannot Be Extended

```
final class ImmutableString {
    private final String value;
```

```
    ImmutableString(String value) {
        this.value = value;
    }

    String getValue() { return value; }
}

class SubString extends ImmutableString { } // COMPILE ERROR!
// Error: "The type SubString cannot subclass the final class ImmutableString"
```

`String` in Java is itself a `final` class! That's why you can't extend `String`. Java's designers wanted `String` behavior to be completely consistent and never modified by subclasses.

Use `final` on a class when:

- You want to prevent inheritance entirely (security, consistency)
- The class is a utility class that shouldn't be extended
- You want immutability guarantees

---

13. FIELDS/VARIABLES DON'T PARTICIPATE IN POLYMORPHISM

This is the most commonly missed concept in polymorphism.

```
class A {
    int x = 10;
}

class B extends A {
    int x = 20; // This does NOT override A's x
}

public class Main {
    public static void main(String[] args) {
        A a = new B(); // A reference, B object

        System.out.println(a.x); // Prints 10, NOT 20!
    }
}
```

This surprises everyone. The object is `B`. Shouldn't `a.x` give 20?

The rule: Variables are bound based on **reference type**, not actual object type. Methods are bound based on actual object type.

For METHODS: Java looks at actual object in memory → dynamic dispatch
For FIELDS: Java looks at reference type → static binding

So `a.x` → reference type is `A` → gives `A`'s `x` which is 10.

To get B's x value, you need to use a B reference:

```
B b = new B();
System.out.println(b.x); // 20
```

Or you can work around this limitation using methods (getters):

```
class A {
   int getX() { return 10; } // Method
}

class B extends A {
   @Override
   int getX() { return 20; } // Override the method
}

A a = new B();
System.out.println(a.getX()); // 20 - because methods DO participate in polymorphism
```

Why does this inconsistency exist?

Methods are designed to be polymorphic — that's the whole point of OOP. Fields are raw data storage. Making fields polymorphic would cause complex aliasing issues and unpredictable behavior. Java keeps fields simple: what you see is what you get based on reference type.

---

14. EVERYTHING TOGETHER — The Complete Picture

Let's connect all four pillars:

```java
// ENCAPSULATION: Data + Behavior bundled, with access protection
abstract class Animal {
    private String name;     // Private = encapsulated, protected
    private int age;         // Private = encapsulated, protected

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }  // Getter for access

    // ABSTRACTION (low level): How run() works internally is hidden from caller
    // ABSTRACTION (high level): run() is abstract - WHAT is declared, HOW is left to subclasses
    public abstract void run();   // Abstract method = high level abstraction

    // Concrete method shared by all animals
    public void breathe() {
        System.out.println(name + " is breathing");
    }
}

// INHERITANCE: Dog IS-A Animal
class Dog extends Animal {
    public Dog(String name, int age) {
        super(name, age);
    }

    // POLYMORPHISM (runtime): Dog's specific implementation of run()
    @Override
    public void run() {
        System.out.println(getName() + " running on four legs");
    }
}

// INHERITANCE: Duck IS-A Animal
class Duck extends Animal {
    public Duck(String name, int age) {
        super(name, age);
    }

    // POLYMORPHISM (runtime): Duck's specific implementation of run()
    @Override
    public void run() {
```

```
        System.out.println(getName() + " waddling");
    }

    // POLYMORPHISM (compile time / overloading): multiple run methods
    public void run(String destination) {
        System.out.println(getName() + " waddling towards " + destination);
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a;

        // Runtime polymorphism - decided at runtime
        a = new Dog("Rex", 3);
        a.run();     // "Rex running on four legs"
        a.breathe(); // "Rex is breathing"

        a = new Duck("Donald", 2);
        a.run();     // "Donald waddling"
        a.breathe(); // "Donald is breathing"

        // Compile time polymorphism
        Duck d = new Duck("Daffy", 4);
        d.run();            // Calls run() - no params
        d.run("the pond");  // Calls run(String) - compile time decision
    }
}
```

---

15. FINAL SUMMARY TABLE

Concept	What it is	How Java implements it	Key Rule
Abstraction	Focus on WHAT, hide HOW	Regular classes, Abstract classes, Interfaces	Separate interface from implementation
Encapsulation	Bundle + Protect data	`private` fields + `public` methods	Data security through access control

Inheritance	IS-A relationship, code reuse	`extends` keyword	Child gets parent's public/protected members
Polymorphism	Many forms, same interface	Method Overloading + Overriding	One call, multiple possible behaviors
Low Level Abstraction	Hide implementation details	Regular classes	User calls method, doesn't need to know internals
High Level Abstraction	Separate WHAT from HOW	Abstract classes + Interfaces	Same call, different actual behavior
Abstract Class	Partial implementation	`abstract class` keyword	Can't instantiate; subclasses must implement abstract methods
Interface	Pure contract/roles	`interface` keyword	All methods abstract by default; use `implements`
Compile Time Polymorphism	Same name, different params	Method Overloading	Decision at compile time based on method signature
Runtime Polymorphism	Same name, different objects	Method Overriding	Decision at runtime based on actual object
Static methods	Belong to class	`static` keyword	NOT overridden; bound to reference type
Final methods	Cannot be changed	`final` keyword	Cannot be overridden by subclasses
Final class	Cannot be extended	`final class`	No subclasses allowed
Fields	Not polymorphic	N/A	Bound to reference type, not object type

This covers every single concept from the video, explained from first principles to implementation, with the reasoning behind every rule.